

Computer Organization, Spring 2026

Project Assignment 3

Pipelined Processor CPU Design

Lecturer: 陳雅淑教授

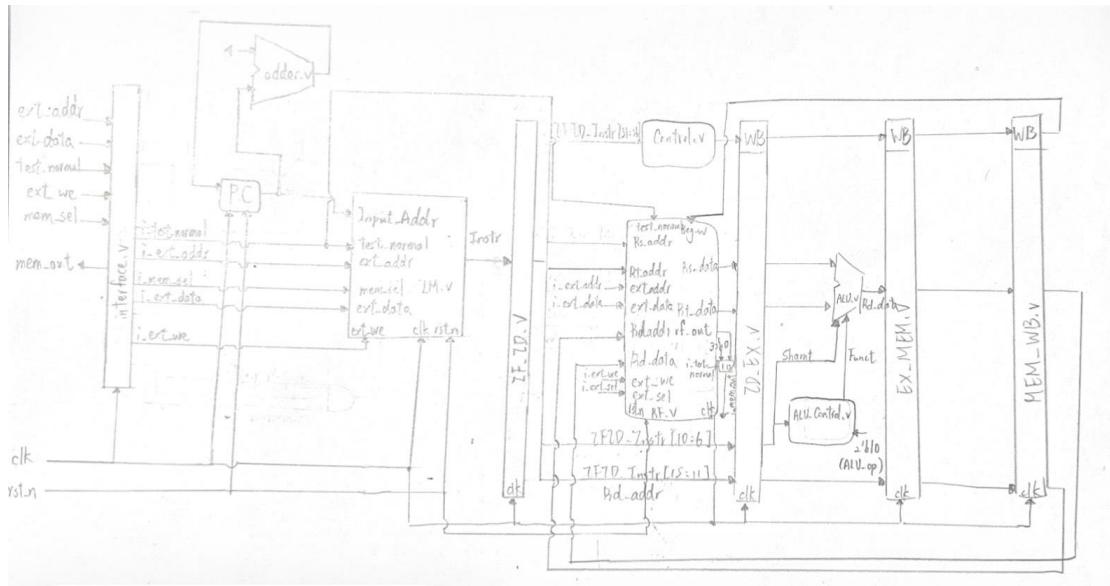
Department: Electrical Engineering

Student ID: B11230024

Name: 王映媛

1.Design Process

Part1.R_PipelineCPU



(a)Adder.v(b)IM.v(c)RF.v(d)ALU.v(e)Control.v(f) ALU_Control.v 同 PA2 part1 功能。

IM.v 與 RF.v：兩個模組具備 Normal Mode 與 Test Mode，由 test_normal 訊號進行切換，我並未在 Top Module 放置多餘的多工器，而是將位址與資料源的判斷 (if-else) 直接內建於模組中，將 MUX 內部化。此外在 RF 模組中，特別開闢一條 rf_out 專屬資料路徑供 Testbench 讀取，避免與 CPU 內部的 Rs/Rt 讀取埠發生資源衝突。

(g)Interface.v 為了在後續合成的時候，input delay 跟 output delay 不會差太多，沒有 Interface.v，wns 會是很大的負值，每一個 posedge clk 將外部 Testbench 傳來的訊號打一拍後輸出成 i_開頭的訊號

(h) IF_ID.v：鎖存從 IM 讀出的 32 位元指令碼。

(i) ID_EX.v：鎖存自 RF 讀出的 Rs/Rt 資料、目標暫存器位址 (Rd)、位移量 (Shamt)、功能碼 (Funct_ctrl)，以及寫回階段的控制訊號 (Reg_w)。

(j) EX_MEM.v：鎖存 ALU 計算結果 (Rd_data)、目標暫存器寫入位址 (Rd_addr)，以及 WB 階段的控制訊號 (Reg_w)。

(k) MEM_WB.v：鎖存 ALU 計算結果 (Rd_data)、目標暫存器寫入位址 (Rd_addr)，以及 WB 階段的控制訊號 (Reg_w)。

1. IF 階段

- Datapath：PC 暫存器輸出當前位址給 IM，IM 讀出 32-bit 指令。同時，加法器將 $PC + 4$ 算好，準備在下個週期更新 PC。
- 傳遞：32-bit 指令存入 IF_ID 暫存器。

2. ID 階段

- Datapath：利用 IF_ID 傳來的 Rs (Instr[25:21]) 與 Rt (Instr[20:16]) 讀取 RF，取得資料、提取出目標暫存器位址 Rd_addr (Instr[15:11])、位移量 Shamt 與功能碼 Funct。
- Control Path：Control.v 根據 OpCode 解碼出控制訊號 Reg_w。
- 傳遞：讀出的資料與提取出的欄位，連同剛剛產生的控制訊號 Reg_w，一併打包存入 ID_EX 暫存器。

3. EX 階段

- Datapath：ALU 接收來自 ID_EX 的源資料與控制碼，計算出結果。
- Control Path：Reg_w 訊號在此階段不被使用。
- 傳遞：ALU 的運算結果(Rd_data)、目標位址 (Rd_addr)，以及 Reg_w 訊號，存入 EX_MEM 暫存器。

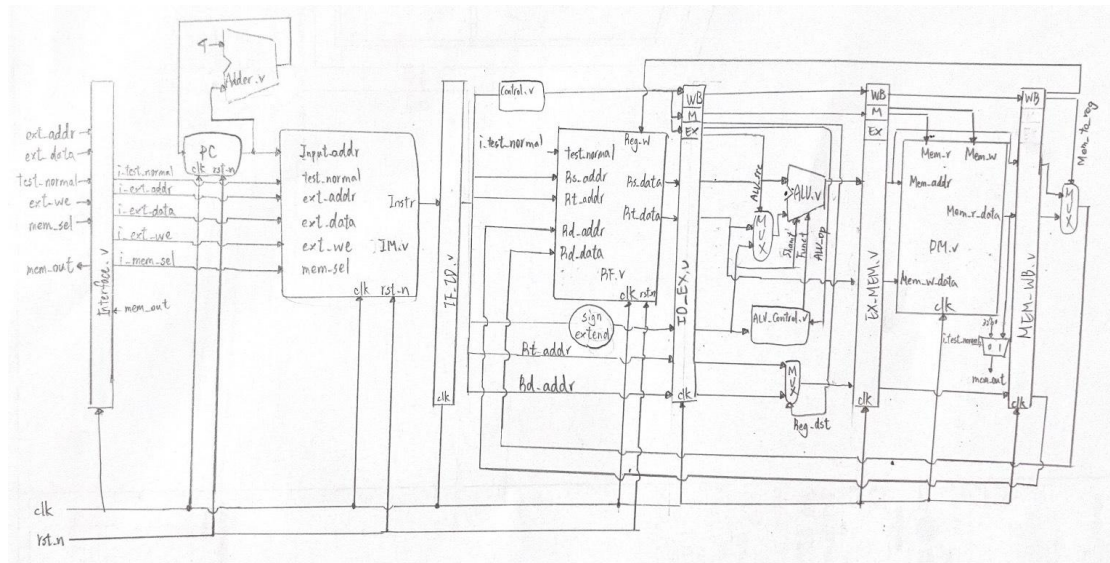
4. MEM 階段

- Datapath & Control Path：由於 part1 僅實作 R-format 指令，不涉及 Data Memory 的讀寫，因此此階段僅傳遞所有的資料（ALU 結果、目標位址）與控制訊號 (Reg_w) 直接存入最後的 MEM_WB 暫存器。

5. WB 階段

Top Module 將控制路徑訊號 MEMWB_Reg_w 及、MEMWB_Rd_addr、MEMWB_Rd_data 拉回 ID 階段的 Register File 寫入埠。因為使用了經過 Pipeline Registers 傳遞下來的 Rd_addr，確保了 ALU 計算出來的結果，寫入最初觸發這條指令的正確暫存器中，避免了因時序錯置而將資料寫入 \$0 或後續指令暫存器的 Hazard 問題。

Part2.I_PipelineCPU



(a)Adder.v(b)IM.v (c)RF.v(d)ALU.v(e) ALU_Control.v(f) Control.v(g) DM.v 同 PA2 part2 的寫法與功能

(h)IF_ID.v：鎖存從 IM 讀出的 32 位元指令碼。

(i)ID_EX.v：鎖存 Rs/Rt 資料、擴展後的立即數、Rs/Rt/Rd 暫存器位址、Shamt 以及解碼出的所有控制訊號。

(j)EX_MEM.v：鎖存 ALU 計算結果、Rt 暫存器原始資料（用於 sw 寫入記憶體）、目標暫存器寫入位址，以及 MEM 與 WB 階段的控制訊號。

(k)MEM_WB.v：鎖存自 DM 讀出的資料、ALU 計算結果、目標暫存器寫入位址，以及 WB 階段的控制訊號。

(l)interface.v 同 part1

1. IF 階段 (Instruction Fetch)

- Datapath：PC 暫存器輸出當前位址給 IM，IM 讀出 32-bit 指令。同時，加法器將 $PC + 4$ 算好，準備在下個週期更新 PC。
- 傳遞：32-bit 指令存入 IF_ID 暫存器。

2. ID 階段 (Instruction Decode)

- Datapath：利用 IF_ID 傳來的 Rs (Instr[25:21]) 與 Rt (Instr[20:16]) 讀取 RF，取得暫存器資料。同時，取出 Rd_addr (Instr[15:11])、Shamt 與 Funct_ctrl。根據 OpCode 將 16-bit (Instr[15:0])進行 Extension，轉換為 32-bit 的 Ext_Imm。

- Control Path：Control.v 根據 OpCode 解碼出本指令在後續各階段所需的所有控制訊號：Reg_dst, Reg_w, ALU_src, ALU_op, Mem_w, Mem_r, Mem_to_reg。
- 傳遞：讀出的 Rs/Rt 資料、擴展後的 Ext_Imm、提取出的各欄位位址/控制碼，連同剛剛產生的所有控制訊號存入 ID_EX 暫存器。

3. EX 階段 (Execute)

- Datapath：
 1. Reg_dst 控制 MUX，決定目標寫入位址是 Rd 還是 Rt
 2. ALU_src 控制 MUX，決定 ALU 第二輸入是 Rt_data 還是 Ext_Imm
 3. ALU 接收源資料與控制碼（由 ALU_Control 產生），計算出 ALU_result
- Control Path：ALU_Control 根據 ALU_op 與 Funct_ctrl 產生 Funct 給 ALU。
- 傳遞：ALU 的運算結果、Rt_data、目標寫入位址，MEM 與 WB 控制訊號 (Mem_w, Mem_r, Mem_to_reg, Reg_w)，存入 EX_MEM 暫存器。

4. MEM 階段 (Memory Access)

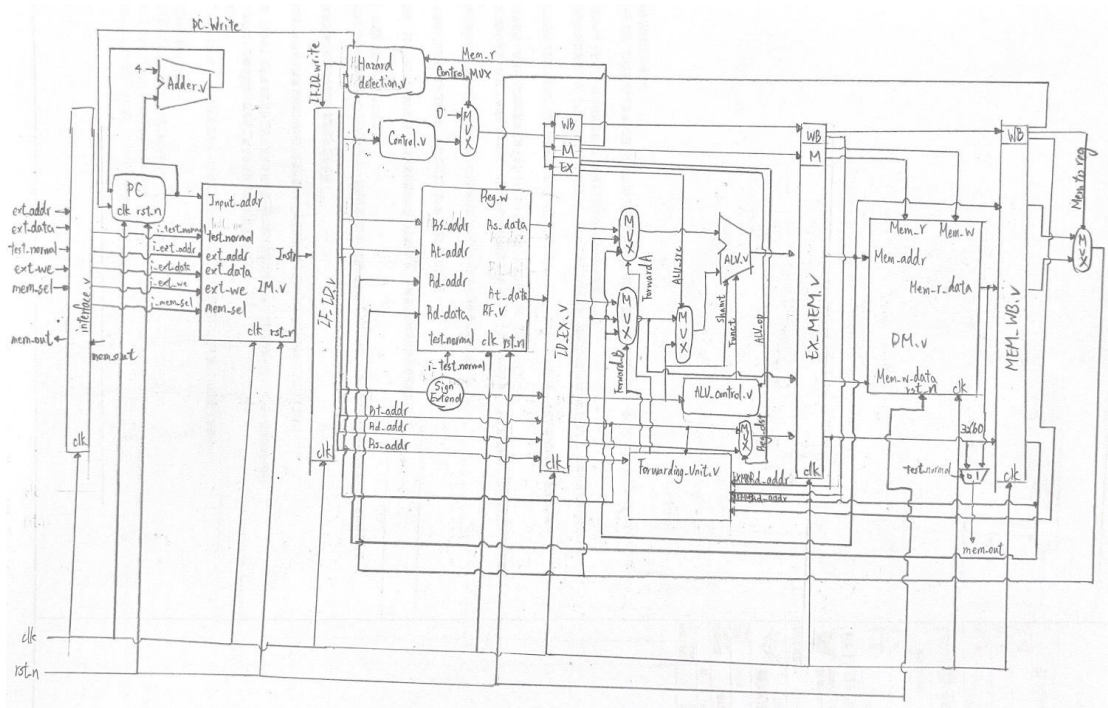
- Datapath：若為 lw，利用 ALU 算出的位址讀取 DM，取得 Mem_r_data；若為 sw，則將傳遞下來的 Rt_data 寫入 DM 中 ALU 算出的位址。
- Control Path：Mem_w 與 Mem_r 訊號在此階段被 DM 使用。
- 傳遞：DM 讀出的資料、ALU 結果、目標寫入位址，WB 控制訊號 (Mem_to_reg, Reg_w)，存入 MEM_WB 暫存器。

5. WB 階段 (Write Back)

- Datapath：Mem_to_reg 控制最後一個 MUX，決定寫回 RF 的資料是來自 DM 還是 ALU。
- Control Path：I_PipelineCPU.v 將控制路徑訊號 MEMWB_Reg_w 及決定的寫回資料與位址 MEMWB_Rd_addr，拉回 ID 階段 RF.v 寫入埠。因為使用了經過 Pipeline Registers 一路傳遞下來的目標位址，確保了最終

資料能寫入最初觸發這條指令的正確暫存器中。

Part3.SimpleCPU



(a) Adder.v、(b) IM.v、(c) RF.v、(d) ALU.v、(e) ALU_Control.v、(f) Control.v、
(g) DM.v、(l) Interface.v：同 part2 的寫法與功能。

(h) IF_ID.v：鎖存從 IM 讀出的 32 位元指令碼。加入 IF_ID_write 腳位，發生 Hazard 時可暫停 Stall。

(i) ID_EX.v：鎖存 Rs/Rt 資料、擴展後的立即數、Rs/Rt/Rd 暫存器位址、Shamt 以及解碼出的所有控制訊號。

(j) EX_MEM.v：鎖存 ALU 計算結果、Rt 暫存器準備寫入記憶體的资料（已包含 Forwarding 最新值）、目標暫存器寫入位址，以及 MEM 與 WB 階段的控制訊號。

(k) MEM_WB.v：鎖存自 DM 讀出的資料、ALU 計算結果、目標暫存器寫入位址，以及 WB 階段的控制訊號。

(m) Forwarding_Unit.v：比較 EX 與 MEM 階段的目標寫回位址 (Rd) 是否與當前 EX 階段的來源位址 (Rs, Rt) 重疊，並負責處理 Double Data Hazard 的優先權，輸出前饋控制訊號。

(n) Hazard_Detection.v：偵測 Load-Use Data Hazard。若 ID/EX 階段為讀取記憶

體指令(lw)且目標位址與當前解碼指令的來源位址重疊，則輸出 Stall 訊號。

(o) FinalCPU.v：頂層模組 (Top module)，負責將上述所有元件與多工器 (MUX) 實體化並完成管線化的 Datapath 與 Control Path 連線。

1. IF 階段 (Instruction Fetch)

- Datapath：PC 暫存器輸出當前位址給 IM，IM 讀出 32-bit 指令。同時，加法器將 $PC + 4$ 算好，準備在下個週期更新 PC。PC 的更新受 Hazard_Detection 發出的 PC_Write 訊號控制；若發生 Load-Use Hazard，PC_Write 為 0，PC 保持原值不變。
- 傳遞：32-bit 指令準備存入 IF_ID 暫存器。存入動作受 IF_ID_write 訊號控制，若發生 Stall，則 IF_ID 不更新，持續鎖存上一道指令。

2. ID 階段 (Instruction Decode)

- Datapath：利用 IF_ID 傳來的 Rs (Instr[25:21]) 與 Rt (Instr[20:16]) 讀取 RF，取得暫存器資料。同時，取出 Rd_addr (Instr[15:11])、Shamt 與 Funct_ctrl。根據 OpCode 將 16-bit 進行 Sign Extension，轉換為 32-bit 的 Ext_Imm。此外，Hazard_Detection 會在此階段即時比對 ID/EX 的 Rt 位址與 IF/ID 的 Rs/Rt 位址。
- Control Path：Control.v 根據 OpCode 解碼出本指令後續所需的控制訊號。若 Hazard_Detection 判斷需暫停管線，會將 Control_MUX 設為 1，使進入 ID_EX 暫存器的關鍵控制訊號（如 Reg_w, Mem_w）強制清零，等同於在管線中插入一個 NOP 指令。
- 傳遞：讀出的 Rs/Rt 資料、擴展後的 Ext_Imm、提取出的各欄位位址 (Rs, Rt, Rd)/控制碼，連同多工器篩選後的控制訊號，存入 ID_EX 暫存器。

3. EX 階段 (Execute)

- Datapath：Forwarding_Unit 根據後段管線傳回的 EXMEM_Rd_addr, MEMWB_Rd_addr 與寫入控制訊號，控制 ALU 前方的 Forward_A 與 Forward_B 3-to-1 MUX。此 MUX 決定 ALU 的輸入資料是來自 ID_EX 原始資料、EX/MEM 的旁路前饋，還是 MEM/WB 的旁路前饋。Reg_dst 控制 MUX，決定目標寫入位址是 Rd 還是 Rt。ALU_src

控制 MUX，決定 ALU 第二輸入是前饋後的 Rt_data(alu_input_b_temp) 還是 Ext_Imm。ALU 接收最終來源資料與控制碼，計算出 ALU_result。

- Control Path：ALU_Control 根據 ALU_op 與 Funct_ctrl 產生 Funct 給 ALU。
- 傳遞：ALU 的運算結果、經過前饋取得的新 Rt_data（用於後續 sw 指令）、目標寫入位址，以及 MEM 與 WB 控制訊號，存入 EX_MEM 暫存器。

4. MEM 階段 (Memory Access)

- Datapath：若為 lw，利用 ALU 算出的位址讀取 DM，取得 Mem_r_data；若為 sw，則將 EX 階段前饋並一路傳遞下來的最新的 Rt_data (EXMEM_Rt_data) 寫入 DM 中 ALU 算出的位址。
- Control Path：Mem_w 與 Mem_r 訊號在此階段被 DM 使用。
- 傳遞：DM 讀出的資料、ALU 結果、目標寫入位址，以及 WB 控制訊號，存入 MEM_WB 暫存器。

5. WB 階段 (Write Back)

- Datapath：Mem_to_reg 控制最後一個 MUX，決定寫回 RF 的資料 (WB_Rd_data) 是來自 DM 還是 ALU。
- Control Path：FinalCPU.v 將控制訊號 MEMWB_Reg_w 及決定的寫回資料與位址 MEMWB_Rd_addr，拉回 ID 階段的 RF.v 寫入埠。同時，這些寫回的資料與位址也會被即時拉回 EX 階段，供 Forwarding_Unit 進行 MEM Hazard 的判斷與前饋。

2. Behavioral Simulation

i. 選定 Load-Use Hazard 偵測與 Forwarding 機制作為最具代表性的測試案例。

此測試同時驗證了：

1. Hazard Detection Unit 的 Stall（管線暫停）與 NOP 插入邏輯。
2. Forwarding Unit 的跨階段資料前饋（Bypass）邏輯。
3. Datapath 的 PC 與 IF/ID 暫存器凍結（Freeze）機制。

指令： lw \$5, 0(\$0)

addi \$6, \$5, 5

解碼與偵測：

- OpCode = 100011 (lw) => Control 設定 Mem_r = 1, Reg_w = 1
- Hazard 條件成立：ID_EX_Mem_r=1，且 ID_EX_Rt_addr(\$5) == IF_ID_Rs_addr(\$5)

Control Path (Hazard 處理)：

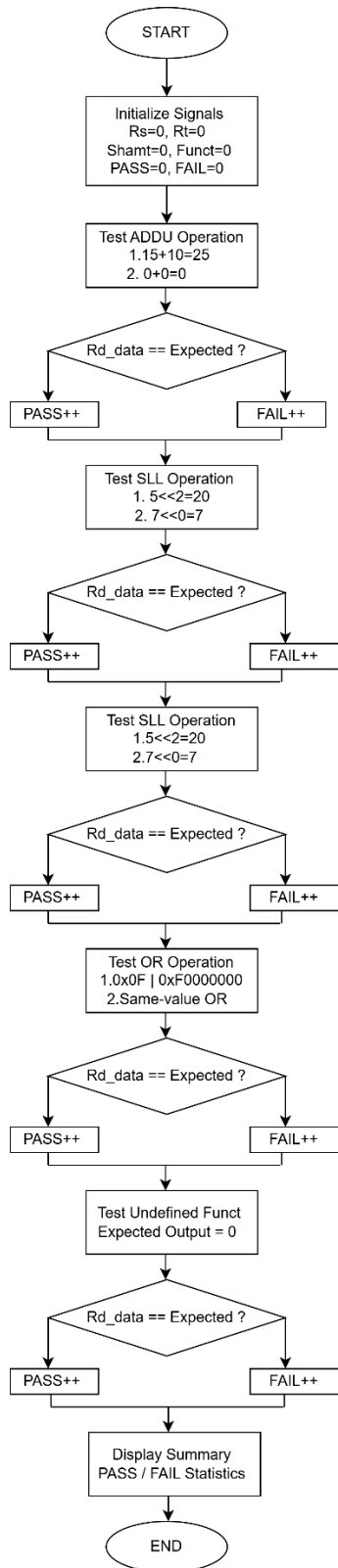
- PC_Write = 0 及 IF_ID_write = 0 => 暫停更新 PC 與 IF/ID，達成管線 Stall。
- Control_MUX = 1 => 將進入 ID/EX 的控制訊號清零，等同插入 NOP。

Datapath (Forwarding 前饋)：

- 暫停一週期後，lw 推進至 MEM 階段讀出資料 0x00000000，addi 進入 EX 階段。
- 條件成立：MEM_WB_Reg_w = 1 且 MEM_WB_Rd_addr(\$5) == ID_EX_Rs_addr(\$5)
- Forwarding_Unit 輸出 Forward_A = 01 (選擇來自 WB 階段的資料)。
- ALU 來源選定：Src_1 = WB_Rd_data = 0 (取代 RF 讀出的舊值)、Src_2 = Ext_Imm = 5。
- ALU 計算： $0 + 5 = 5$ 。
- 後續的 sll 指令亦成功觸發 EX Hazard (Forward_B = 10)，取得 5 並移位算出 20，最終由 sw 存入記憶體。

ii. ALU_tb.v 檢測 ALU 是否確實運作

(a) flow chart



(b) ALU_tb.v

```

`timescale 1ns/1ps

module tb_ALU;
  reg [31:0] Rs_data;
  reg [31:0] Rt_data;
  reg [4:0] Shamt;
  reg [5:0] Funct;
  wire [31:0] Rd_data;
  ALU uut (
    .Rs_data (Rs_data),
    .Rt_data (Rt_data),
    .Shamt (Shamt),
    .Funct (Funct),
    .Rd_data (Rd_data)
  );
  integer pass_count;
  integer fail_count;
  task check;
    input integer test_id;
    input [31:0] expected;
    input [8+30:1] test_name;
    begin
      #5;
      if (Rd_data == expected) begin
        $display(" [PASS] Test %0d | %s | Rs=%0d Rt=%0d Shamt=%0d Funct=%06b => 0x%08X",
          test_id, test_name, Rs_data, Rt_data, Shamt, Funct, Rd_data);
        pass_count = pass_count + 1;
      end else begin
        $display(" [FAIL] Test %0d | %s | Rs=%0d Rt=%0d Shamt=%0d Funct=%06b",
          test_id, test_name, Rs_data, Rt_data, Shamt, Funct);
        $display(" Expected: 0x%08X Got: 0x%08X", expected, Rd_data);
        fail_count = fail_count + 1;
      end
    end
  endtask
  initial begin
    pass_count = 0; fail_count = 0;
    Rs_data = 0; Rt_data = 0; Shamt = 0; Funct = 0;
    $display("=====");
    $display("          ALU Testbench Start");
    $display("=====");
    #10;
    // =====
    // Group 1: Addu (Funct = 001001)
    // =====
    $display("\n(Group 1) Addu (Funct=001001)");
    Rs_data=32'd15; Rt_data=32'd10;
    Shamt=0; Funct=6'b001001;
    check(1, 32'd25, "15+10=25");
    Rs_data=32'd0; Rt_data=32'd0;
    check(2, 32'd0, "0+0=0");

    // =====
    // Group 2: Subu (Funct = 001010)
    // =====
    $display("\n(Group 2) Subu (Funct=001010)");
    Rs_data=32'd20; Rt_data=32'd8;
    Shamt=0; Funct=6'b001010;
    check(5, 32'd12, "20-8=12");
    // BEQ taken 情境：兩數相等，結果為0
    Rs_data=32'd15; Rt_data=32'd15;
    check(6, 32'd0, "15-15=0 (BEQ taken)");

    // =====
    // Group 3: Sll (Funct = 100001)
    // =====
    $display("\n(Group 3) Sll (Funct=100001)");
    Rs_data=32'd0; Rt_data=32'd5; Shamt=5'd2; Funct=6'b100001;
    check(9, 32'd20, "5<<2=20");
    Rs_data=32'd0; Rt_data=32'd7; Shamt=5'd0;
    check(10, 32'd7, "7<<0=7");

    // =====
    // Group 4: OR (Funct = 100101)
    // =====
    $display("\n(Group 4) OR (Funct=100101)");
    Rs_data=32'h0000000F; Rt_data=32'hF0000000;
    Shamt=0; Funct=6'b100101;
    check(13, 32'hF000000F, "0x0F|0xF0000000");
    Rs_data=32'hABCDEF01; Rt_data=32'hABCDEF01;
    check(14, 32'hABCDEF01, "same value OR");

    // =====
    // Group 5: Default (未定義 Funct)
    // =====
    $display("\n(Group 5) Default (undefined Funct)");
    Rs_data=32'hDEADBEEF; Rt_data=32'h12345678;
    Shamt=0; Funct=6'b111111;
    check(17, 32'd0, "undefined->0");
    $display("\n=====");
    $display(" Result: PASS=%0d FAIL=%0d Total=%0d",
      pass_count, fail_count, pass_count+fail_count);
    if (fail_count == 0)
      $display(" All tests PASSED. ALU is correct.");
    else
      $display(" %0d test(s) FAILED. Please check ALU.v", fail_count);
    $display("=====");
    $finish;
  end
endmodule

```

(c)tcl console

```

=====
          ALU Testbench Start
=====

[Group 1] Addu (Funct=001001)
[PASS] Test 1 |                               15+10=25 | Rs=15 Rt=10 Shamt=0 Funct=001001 => 0x00000019
[PASS] Test 2 |                               0+0=0 | Rs=0 Rt=0 Shamt=0 Funct=001001 => 0x00000000

[Group 2] Subu (Funct=001010)
[PASS] Test 5 |                               20-8=12 | Rs=20 Rt=8 Shamt=0 Funct=001010 => 0x0000000c
[PASS] Test 6 |                               15-15=0 (BEQ taken) | Rs=15 Rt=15 Shamt=0 Funct=001010 => 0x00000000

[Group 3] Sll (Funct=100001)
[PASS] Test 9 |                               5<<2=20 | Rs=0 Rt=5 Shamt=2 Funct=100001 => 0x00000014
[PASS] Test 10 |                              7<<0=7 | Rs=0 Rt=7 Shamt=0 Funct=100001 => 0x00000007

[Group 4] OR (Funct=100101)
[PASS] Test 13 |                   0x0F|0xF0000000 | Rs=15 Rt=4026531840 Shamt=0 Funct=100101 => 0xf000000f
[PASS] Test 14 |                   same value OR | Rs=2882400001 Rt=2882400001 Shamt=0 Funct=100101 => 0xabcdef01

[Group 5] Default (undefined Funct)
[PASS] Test 17 |                   undefined0 | Rs=3735928559 Rt=305419896 Shamt=0 Funct=111111 => 0x00000000

=====
Result: PASS=9 FAIL=0 Total=9
All tests PASSED. ALU is correct.

```

(d)simulation waveforms

Name	Value	0.000 ns	5.000 ns	10.000 ns	15.000 ns	20.000 ns	25.000 ns	30.000 ns	35.000 ns	40.000 ns	45.000 ns	50.000 ns
> Rs_data[31:0]	deadbeef	00000000	0000000f	00000000	00000014	0000000f	00000000	00000000	0000000f	abcdef01	deadbeef	
> Rt_data[31:0]	12345678	00000000	0000000a	00000000	00000008	0000000f	00000005	00000007	f0000000	abcdef01	12345678	
> Shamt[4:0]	00			00				02			00	
> Funct[5:0]	3f	00		09		0a		21		25		3f
> Rd_data[31:0]	00000000	00000000	00000019	00000000	0000000c	00000000	00000014	00000007	f000000f	abcdef01	00000000	
> pass_count[31:0]	00000009		00000000		00000001	00000002	00000003	00000004	00000005	00000006	00000007	00000008
> fail_count[31:0]	00000000						00000000					

(e) explanation of the design

- **計數器**：使用了兩個整數變數 `pass_count` 和 `fail_count`，用來追蹤測試通過與失敗的總次數。
- **比對任務**：定義了一個名為 `check` 的任務，接收測試編號、預期結果 (expected) 以及測試名稱作為參數。
- **判定邏輯**：在延遲 5 個時間單位後，程式會比對實際的 `Rd_data` 是否與 `expected` 完全相同。
- **輸出報告**：如果結果相符，系統會印出 [PASS] 的詳細資訊並將 `pass_count` 加 1；如果不符，則會印出 [FAIL]、預期數值與實際收到的數值，並將 `fail_count` 加 1。

Test Scenarios:

在 `initial` 區塊中，依序對 ALU 送出了五個群組的測試案例，模擬處理器中常見的指令行為：

- **Group 1 (Addu)**：測試無號數加法 (Funct = 001001)，驗證 $15 + 10 = 25$ ，以及 $0 + 0 = 0$ 。
- **Group 2 (Subu)**：測試無號數減法 (Funct = 001010)，例如驗證 $20 - 8 = 12$ ，以及 $15 - 15 = 0$ 。
- **Group 3 (Sll)**：測試邏輯左移 (Funct = 100001)，將 `Rt_data` 的值 (5) 依照 `Shamt` 指定的位數 (2) 向左平移，預期結果為 20。
- **Group 4 (OR)**：測試位元邏輯或運算 (Funct = 100101)，驗證 `0x0000000F` 與 `0xF0000000` 的合併結果。
- **Group 5 (Default)**：測試未定義的例外功能碼 (Funct = 111111)，預期 ALU 應具備防呆機制 (Default case) 並直接輸出 0。

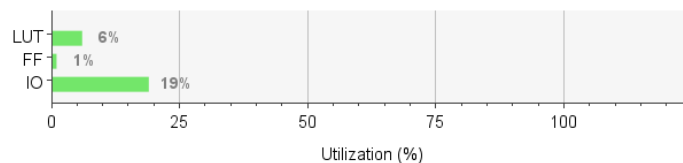
測試總結：

在所有指令測試完畢後，程式碼會印出總結儀表板，顯示總共的 PASS 次數、FAIL 次數與測試總數。如果 fail_count 為 0，會顯示所有測試皆通過 (All tests PASSED. ALU is correct.)。如果大於 0，則會警告使用者有測試失敗，並提示去檢查 ALU.v 原始檔。

結束模擬：透過 \$finish 指令終止系統的模擬程序。

3. Synthesis Result (part 3)

Resource	Utilization	Available	Utilization %
LUT	7673	134600	5.70
FF	3513	269200	1.30
IO	76	400	19.00



Hardware Utilization:

LUT: 7673 / 134600 (5.7%)。LUT 主要用於設計中的組合邏輯(ALU 算術與邏輯運算、Control Unit 的指令解碼、Forwarding Unit 與 Hazard Detection 的位址比對邏輯，以及各個階段的資料多工器 (MUX) 都會消耗 LUT 資源。

FF (Flip-Flop): 3513 / 269200 (1.30%)。FF 主要用於實作循序邏輯(具有記憶功能的元件: PC、RF 內部的暫存器陣列 IF/ID, ID/EX, EX/MEM, MEM/WB)。

IO (Input/Output): 76 / 400 (19%)。

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 43.491 ns	Worst Hold Slack (WHS): 0.146 ns	Worst Pulse Width Slack (WPWS): 49.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 9881	Total Number of Endpoints: 9881	Total Number of Endpoints: 3514

All user specified timing constraints are met.

Timing Report:

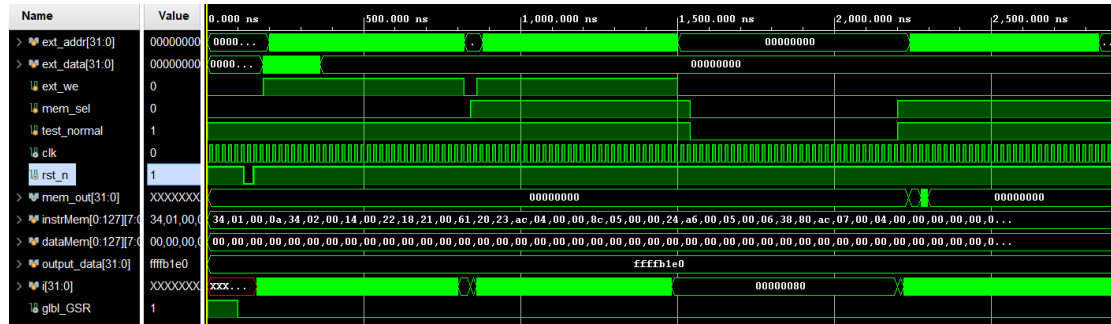
Setup Time : WNS: 43.491 ns TNS: 0.000 ns

Hold Time: WHS: 0.146 ns THS: 0.000 ns WHS，成功保持了足夠長的時間讓正反器穩定取樣，沒有發生 Race Condition。

Pulse Width : WPWS: 49.5 ns，顯示時鐘訊號的高低電位寬度完全符合硬體元件的物理切換限制。

4. Post-Synthesis Timing Simulation (part 3)

1. hexadecimal waveform



2. Vivado Tcl 控制台分析

```
--- [Data Memory Results] ---
DM[ 0] : 00000014
DM[ 4] : 00000064
DM[ 8] : 00000000
DM[12] : 00000000
DM[16] : 00000000
DM[20] : 00000000
DM[24] : 00000000
DM[28] : 00000000
DM[32] : 00000000
DM[36] : 00000000
DM[40] : 00000000
DM[44] : 00000000
DM[48] : 00000000

Simulation Completed. By IEESLab

  /\  /\  /\  /\  Congratulations !
 ( o.o ) ( -. ) ( ^_^ ) Please Check <<DM_test.out>>
 > ^ < > ^ < > ^ < Wishing you continued success and happiness. By IEESLab

$finish called at time : 2900 ns : File "C:/Users/USER/Desktop/PA3/tb_FinalCPU.v" Line 205
xsim: Time (s): cpu = 00:00:03 ; elapsed = 00:00:07 . Memory (MB): peak = 3423.969 ; gain = 0.000
```

時序收斂：在硬體延遲環境下，位址跳轉與數據存取均無時序違規

功能正確：指令執行流程與 DM_test.out 最終運算結果均符合預期

5. Conclusion

在這次做 5-stage pipeline CPU 的專題，驗證了課本上講的 Datapath、Control Path，還有 Data Forwarding 跟 Load-Use Hazard 機制。這次專題我卡關最久、最崩潰是從 Behavioral Simulation Post-Synthesis Timing Simulation。一開始我 IM.dat 指令寫得太短，而且後面完全沒有補零，結果在純軟體邏輯的 Behavioral Simulation 跑出來完全正確，但一跑 Post-Synthesis Timing Simulation 的時候，Data Memory 的輸出直接全部歸零。我嘗試了無數種修正方式，像是瘋狂改 Testbench 的時序、重新調整 XDC 檔的 constraints，甚至懷疑是不是 Vivado 的問題，找助教問一個下午也沒用，整個卡死在原地，最後請別的同学用他的 TB 跟 IM.dat 跑我的程式居然就可以，回家後我試著把 IM.dat 寫完整，即可正常運作，這應該是本學期最艱難的挑戰。